

コンピュータ アーキテクチャ I 第3回

塩谷 亮太 (shioya@ci.i.u-tokyo.ac.jp)

東京大学大学院情報理工学系研究科 創造情報学専攻

もくじ

1. 前回の振り返り
2. 課題の解説
3. 2進数や16進数による数値表現
4. 論理回路によるCPUの実装

前回の振り返り (プログラムと簡単な CPU)

プログラム

- プログラムとは
 - 計算の手順を表したもの
 - 実体：メモリの上にある，計算方法を指示する数字（命令）の列
- （フォン）ノイマン型 (von Neumann-type) コンピュータ
 - プログラムに従って計算をする機械
 - メモリに格納された命令を取り出して順に実行
 - 他にもあるけど，これが今日では主流
- 次項から，簡単な例を使って説明

例 : A + B - C

■ 形式的に表すと :

1. add A, B → D // D は一時的に結果をおいておく変数
2. sub D, C → E // E に A + B - C の結果

■ 数字の列で表してみる :

● 変換の規則 :

意味 :	add	sub	A	B	C	D	E
数字 :	0	1	2	3	4	5	6

● 数列 :

1. 0, 2, 3, 5 // add(0) A(2), B(3) → D(5)
2. 1, 5, 4, 6 // sub(1) D(5), C(4) → E(6)

プログラムの表現と用語（1）

- バイナリ： 0, 2, 3, 5, 1, 5, 4, 6
 - 計算方法を表す数字の列
 - コンピュータが直接理解できるのは、このバイナリのみ
- アセンブリ言語： add A, B → D
 - バイナリと1：1に対応しており、基本的に「相互に」変換可能
 - 要はバイナリを人間にとって読みやすくしたもの
- 機械語：
 - 上記のバイナリないしはアセンブリ言語で表現されたプログラム

プログラムの表現と用語（2）

■ 命令：

- コンピュータが解釈できるプログラム内の計算手順の最小単位
- 「0, 2, 3, 5」 「add A, B→D」

■ オPCODE (opcode)

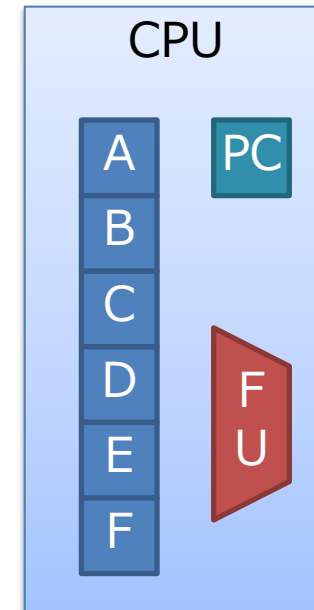
- 命令でどういう計算をするか指定する部分
- 「0, 2, 3, 5」 「add A, B→D」

■ オペランド (operand)

- 計算の入出力対象を指定する部分
- 「0, 2, 3, 5」 「add A, B→D」
- 入力をソース, 出力をディスティネーション とよぶ

CPU

- コンピュータの心臓部
 - メモリから命令を読み出し，計算する
- 構成要素：
 - 演算器（FU: Functional Unit）
 - 加算器や AND 演算器など
 - 指示された種類の演算を行う
 - レジスタ・ファイル（右図では A,B,C...）
 - メモリと同様にデータを記憶する
 - * 位置を指定して読み書きする
 - CPU の演算は，このレジスタ上でのみ行う
 - PC（Program Counter）
 - 現在見ている命令のアドレスを記憶している場所

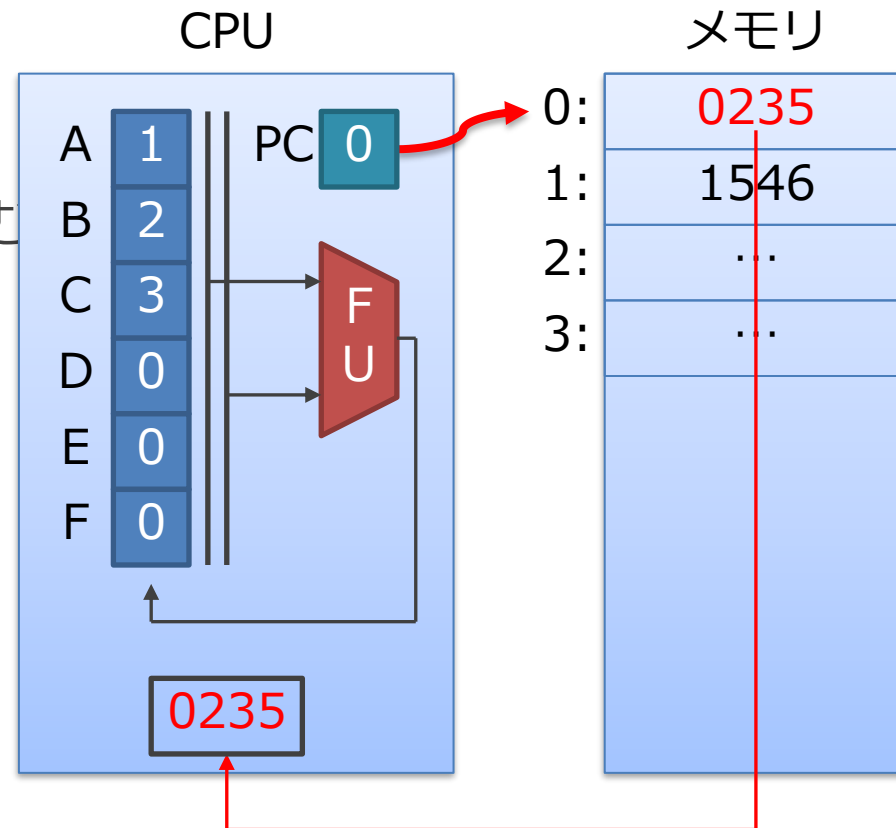


CPU の動作

- PC (Program Counter) :
 - 現在処理する命令のアドレスを保持
- おおざっぱな命令の処理 :
 1. PC が指すアドレスのメモリから読む
 2. 読んできた命令に応じて処理をする
 3. PC を更新 (数字をたす)
 4. 1. にもどる

1. 命令の読み出し（フェッチ）

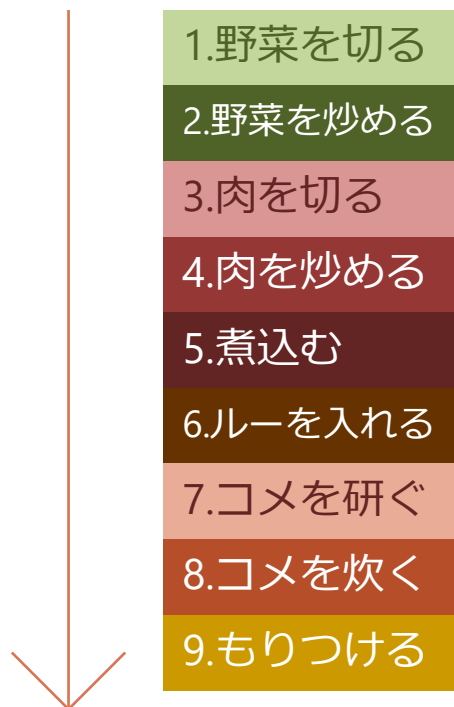
1. PC が指している命令の番地を読む
 1. 今はアドレス 0 を指している
2. 内容である 0235 が得られる
3. CPU 内にもってくる



CPU の動作は料理に似ている

- レシピをみながら料理をするのに似ている
 - レシピの各手順が命令
 - 「今何個目の手順を見てるか」, を憶えているのかが PC
 - ひとつひとつ手順を取り出して, 指示に従って処理

カレーのレシピ



課題 2 の解説

課題2で使用する命令セット

■ レジスタ :

- A, B, C, D, E, F の6つがあるものとする

■ 命令 :

- 次のページで説明する li, add, sub, b の命令がある

課題 2 で使用する命令セット

- li : 即値をレジスタに読み込む
 - 例 : li 0 → A // レジスタ A に 0 を入れる
- add : 加算
 - 例 : add A,1 → A // レジスタ A に 1 を足して A に書く
- sub : 減算
 - 例 : sub B,C → D // レジスタ B から C を引いて D に書く

課題 2 で使用する命令セット

- b : 条件分岐命令

- 例 : `b A < B, LABEL` // もし $A < B$ であれば LABEL に飛ぶ
- 例 : `b LABEL` // 条件式がなければ無条件に LABEL に飛ぶ

課題2で使用する命令セット

■ ラベルについて

- C言語のラベルと同様に、任意の命令の場所を表すためにラベルを使うことができる
- ラベルには任意の名前をつけることが可能であり、「ラベル名：」で表記する

■ 例：

- ```
li A←0
LABEL_EXAMPLE: // LABEL_EXAMPLE というラベルをここに定義
add A←A+A
b LABEL_EXAMPLE // LABEL_EXAMPLE に無条件で飛ぶ
 // 具体的には add に飛ぶ
```



## 課題 2 (1, 2)

- (1) 以下の C 言語で書かれたプログラムをそれぞれアセンブリ言語で表せ.
- (2) アセンブリ言語で書いたそれぞれのプログラムを実行した際の, 実行される命令の系列を時系列で書け.

(a)

```
1: i=4;
2: if (i > 1)
3: i++;
4: i--;
```

(b)

```
1: i=4;
2: if (i <= 1)
3: i--;
4: i++;
```

# 補足 1

- 「 $\text{add } A \leftarrow A + 1$ 」 「 $\text{add } A + 1 \rightarrow A$ 」 「 $\text{add } A, 1 \rightarrow A$ 」 はどれが正しいのか？
  - 意味がわかれば、どれでも良いです
    - このオリジナルのアセンブリ言語は実物ではないので、概念的な意味が表せて通じればそれでいいです
  - とはいえ、こちらの表記が一定していない時があり、すいません

## 補足 2

- LABEL は定義する前に使っても構いません
  - 分岐の後に LABEL が出てきても問題ないです
  - 実は C 言語でも同じで, 以下の様に書けます

```
int main() {
 int i = 1;
 goto LABEL;
 i++;
LABEL:
 return 1;
}
```

# 解答例 (a)

(a)

```
1: i=4;
2: if (i > 1)
3: i++;
4: i--;
```

(1)

```
li A←4
b A≤1, LABEL
add A←A+1
LABEL:
sub A←A-1
```

(2)

```
li A←4
b A≤1, LABEL
add A←A+1
sub A←A-1
```

- if ( $i > 1$ )  $i++$  は、逆に  $i \leq 1$  の時に飛ばすようにするとシンプルに書ける

## 解答例 (b)

(b)

```
1: i=4;
2: if (i <= 1)
3: i--;
4: i++;
```

(1)

```
li A←4
b A>1, LABEL
sub A←A-1
LABEL:
add A←A+1
```

(2)

```
li A←4
b A>1, LABEL
sub A←A-1
```

# 2進数や16進数による数値表現

---

# 2進数や16進数による数値表現

- まず数値表現について軽く説明する
  - 実際の命令セットでは2進数や16進数が多く出てくるため
  - 慣れてないと理解しづらい

# 表記方法

## ■ ここから先では C 言語の表記方法に従う

- 10進数：なにもつけない (143)
- 2進数：先頭に 0b をつける (0b10000011)
- 16進数：先頭に 0x をつける (0x83)

## ■ 由来：

- 0b = binary (英語の 2 進数)
- 0x = hexadecimal (英語の 16 進数)
- bit = binary digit の短縮系

## ■ 備考：

- 0b による 2 進数表記は C23/C++14 規格から導入されたので、古い C 言語の資料では書かれていない



# 現代のコンピュータは基本的に2進数ベースで出来ている

- 電圧が「高い=1」 or 「低い=0」の2進数で表現されている
  - たまに1 or 0の割り当てが逆のこともある
- なぜ2進数なのか？
  - 回路を作るのが簡単だから

# 現代のコンピュータは基本的に2進数ベースで出来ている

- たとえば3進数の場合,
  - 電圧が「高い」「中間」「低い」の3つを区別することになる
  - この判別は、単に高いか低いかよりもかなり難しい
    - (後で実例を紹介)
- 手の指で表した場合でも同じ？
  - 指が「折れている=0」「立っている=1」で2進数を表す
    - $2^{10}=1024$  通りを表す事ができる
  - 3進数にすると「半分折れている」が加わる
    - $3^{10}=59049$  通りを表すことができる
    - しかし、「半分折れてる」の判別がむずかしい

# 情報分野では2進/16進/10進が混じって出てくる事が多い

## ■ 注意：

- これらは単に表記方法の違い
- 表し方が違うだけで，それが示す数字そのものは同じ

## ■ たとえば10進の「12」は3通りに書けるが，意味は同じ

- 10進：12
- 2進：0b1100
- 16進：0xc

## ■ 「リンゴ」を「林檎」「りんご」「Apple」と書いても意味するものは変わらないのと同じ

# ではなぜ複数の表記を使うのか？

- それぞれの表記方に利点があるから
  - 10進は人間が普段使っているので，人間が考えやすい
  - 2進や16進は？

# 2進数で表記する利点

## ■ 前回の講義でだした CPU では10進で命令を定義していた

- 人間に分かりやすくするため
- 10進数の各桁を見れば意味がわかる

1. 0 2 3 5      // add(0) A(2), B(3) → D(5)

2. 1 5 4 6      // sub(1) D(5), C(4) → E(6)

## ■ コンピュータは2進数の形で数字を保持している

- 2進数の各桁に意味を持たす事が多い
- 2進数表記した方が, 色々と考えやすい

# たとえば4ビットのデータを考える

- 4ビットのデータは 0b0000~0b1111 → 0 ~ 15 を表す事ができる
- 2進数で下から3桁目が add/sub を示すとする
  - 2進の場合：3桁目だけを見れば判定できる
    - add : 0bx1xx (x は 0 or 1 の任意)
    - sub : 0bx0xx
  - 10進の場合：かなり意味が分からない
    - add : 4, 5, 6, 7, 12, 13, 14, 15 のいずれか
    - sub : 0, 1, 2, 3, 8, 9, 10, 11 のいずれか

# 2進数の欠点

- 人間にわかりにくい

1. 桁数が大きくなりすぎる
2. 10進でどのぐらいの数字になるのか, 検討をつけにくい

- たとえば

- $0b1101010010000111 = 54407$

# 1 6進数

## ■ 1 6進数

- 1 0進数の0~15を

0x0, 0x1, 0x2, 0x3, 0x4, 0x5, 0x6, 0x7,  
0x8, 0x9, 0xa, 0xb, 0xc, 0xd, 0xe, 0xf で表す

## ■ 利点 :

- 表記上の桁数が少ないので, 2進数よりは人間にわかりやすい
- 2進数と1 6進数は相互変換が簡単



# 2進数と16進数の相互変換

- 2進数の4桁が16進数の1桁に1 : 1 : に直接対応する
  - 2進 : 0b 1101 0100 1000 0111
  - 16進 : 0x D 4 8 7
  - 10進 : 54407
- 2進数の4桁ごとに対応をとれば簡単に変換できる
  - 10進との相互変換はかなり難しい

# (余談) 手計算でやるとき

- 2進→16進の手計算は、以下のように憶えると便利
  - 4桁ずつに区切る
  - 下から見て、1が立っているところに1248をかけて足す
- たとえば,
  - 2進 : 0b 1101 0100 1000 0111
  - 16進 : 0x D      4      8      7
  - 一番上位の4桁は 1101 → 1+4+8=13 → D
  - 次の4桁は 0100 → 4

# なぜ 16 なのか？

- 2 進数との相互変換のしやすさだけなら，8 進や 3 2 進でも良い
  - 2 の累乗の進数なら，同様に各桁が 1 : 1 に対応する
- 16 進数では 1 桁を表すのに 4 ビットが必要
  - このビット数も 2 の累乗の方が何かと都合がよい
    - 8 進数だと 3 ビットになり，2 の累乗から外れる
  - 10 進に近いので，考えやすい

## (余談) 二進化十進表現 (Binary-coded decimal: BCD)

- 2進数の4桁（16進数の1桁）で10進数の各桁を表す方法
  - 0b0000(0x0) ~ 0b1001(0x9) が 0~9 を示す
  - 例：0x1234 は 1234 を示す
- BCD の利点と欠点
  - 利点：桁が 1 : 1 に対応するので 2進数と 10進数の相互変換が楽
  - 問題：0xa ~ 0xf は使わないので同じ桁数で表せる数字の数が減る

- 基本的に無駄
  - なので，現代ではあまり使われない
- ただし，お金を扱う場面では今でも使われる事がある
  - 10進でキリの良い数字が，2進では循環小数になったりする
    - 0.1 は2進だと 0.0001 1001 1001 1001 1001 ...
    - 2進では有限の桁数で表せないなので誤差が出る
  - 税金や利子の計算などは10進数前提でルールが組まれている
    - 消費税 10% のたびに微妙な誤差が出たらまずい
    - 利子が微妙に多くなったり小さくなるのもまずい

# 論理回路による実装

---

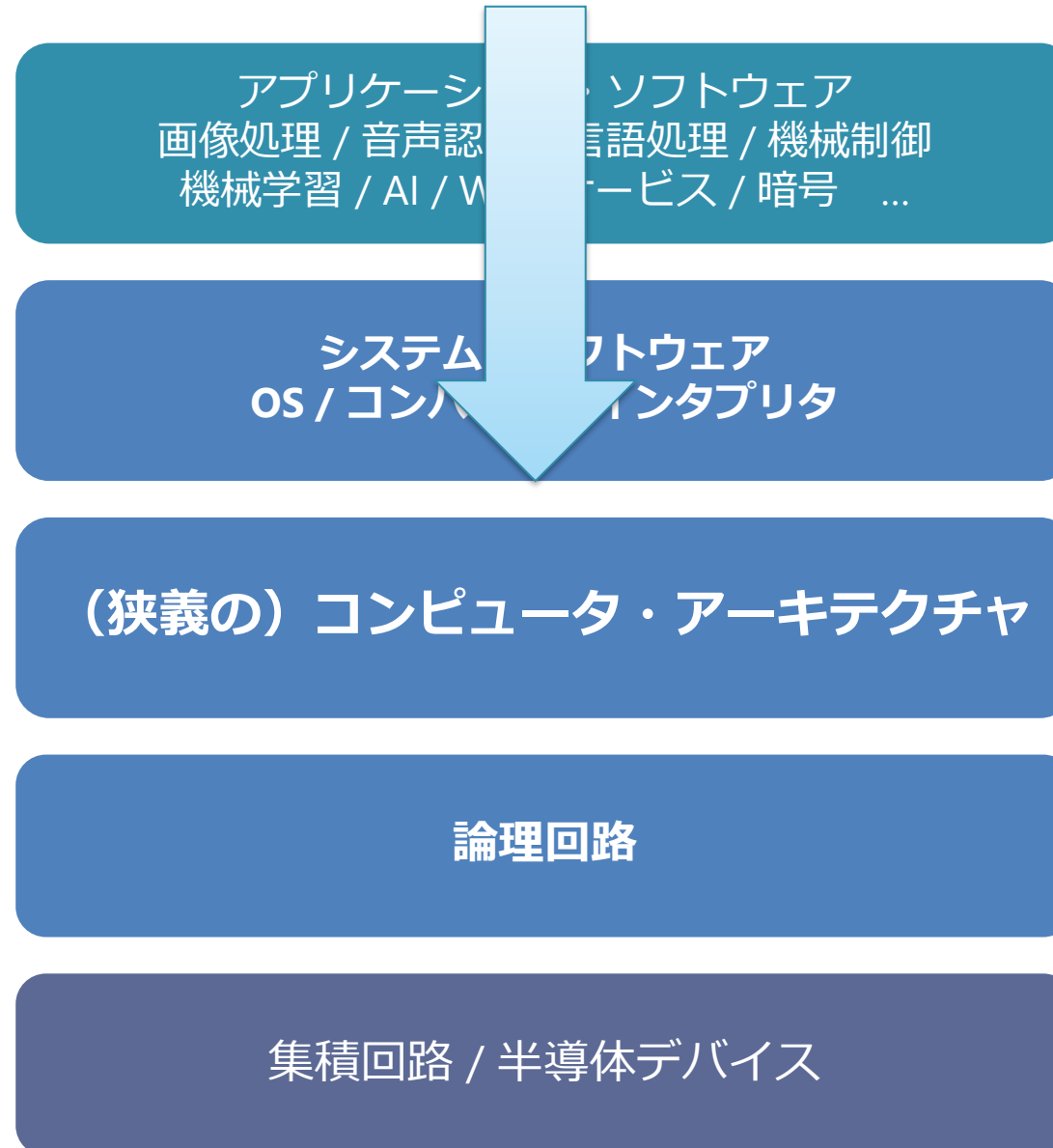
# 回路と遅延

- 目的：これらの具体的なイメージを持つ
  - CPU の論理的な動作と，それを実現する回路の繋がり
  - （それら論理回路の遅延や消費エネルギー
- 論理回路の復習から始めて説明
  - 論理回路

（次回以降：

  - CMOS による実現
  - 遅延と消費電力がどのように決まるのか

前は、「C 言語で書かれたプログラムを動かすためには」という視点で上から迫っていた





今回は、「コンピュータのハードを作るためには」  
という視点で、さらに下がっていく

アプリケーション・ソフトウェア  
画像処理 / 音声認識 / 言語処理 / 機械制御  
機械学習 / AI / WEB サービス / 暗号 ...

システム・ソフトウェア  
OS / コンパイラ / インタプリタ

(狭義の) コンピュータ・アーキテクチャ

論理回路

集積回路 / 半導体デバイス

# 論理回路の復習

---

# 組み合わせ回路と順序回路

## 1. 組み合わせ回路

- 出力が、現在の入力のみにより決定される論理回路

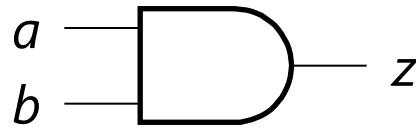
## 2. 順序回路

- 出力が、過去の入力（の履歴）にも依存する論理回路

# 組み合わせ回路の例：2入力論理ゲート

AND  
(論理積)

MIL記号  
MIL symbol



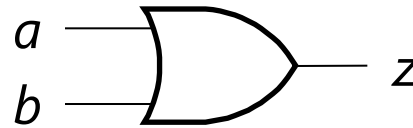
論理式  
logic expression

$$z = a \cdot b$$

真理値表  
truth table

| $a$ | $b$ | $z$ |
|-----|-----|-----|
| 0   | 0   | 0   |
| 0   | 1   | 0   |
| 1   | 0   | 0   |
| 1   | 1   | 1   |

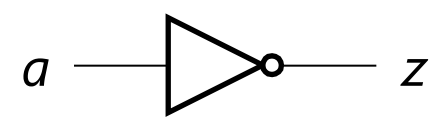
OR  
(論理和)



$$z = a + b$$

| $a$ | $b$ | $z$ |
|-----|-----|-----|
| 0   | 0   | 0   |
| 0   | 1   | 1   |
| 1   | 0   | 1   |
| 1   | 1   | 1   |

NOT  
(論理否定)



$$z = a'$$

$$z = \bar{a}$$

$$z = \neg a$$

| $a$ | $z$ |
|-----|-----|
| 0   | 1   |
| 1   | 0   |

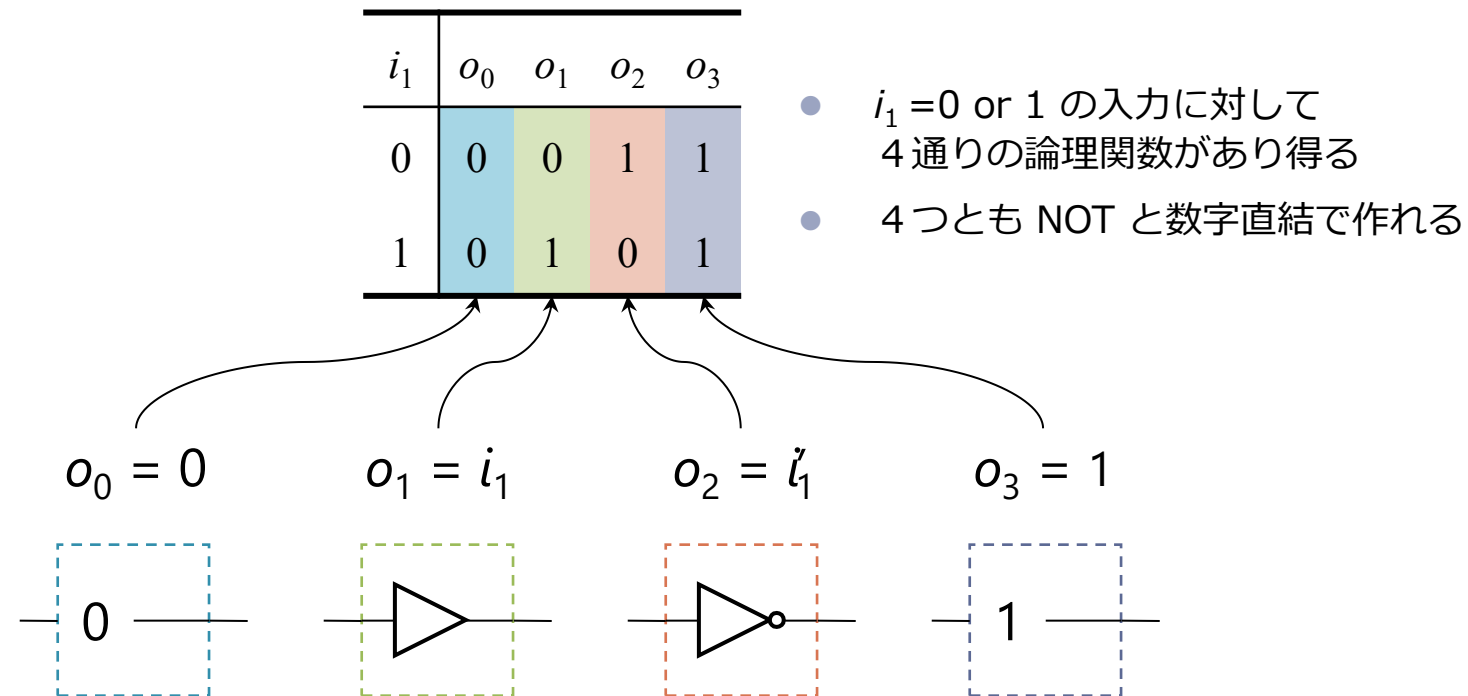
# 完全性 (Completeness, 完備性)

- 完全集合 (Complete Set) :
  - その組み合わせによって, すべての論理関数を表現できる論理関数の集合
- 完全集合の例
  - {AND, OR, NOT}
  - {AND, NOT}
  - {OR, NOT}
  - {NAND}
  - {NOR}
- たとえば, {AND, OR, NOT} を組み合わせると任意の論理関数ができる

# 完全性の証明 {AND, OR, NOT}

## ■ 数学的帰納法：

1. 1入力の論理関数は {AND, OR, NOT} の組合せで表現できる
2.  $n$  入力の関数を {AND, OR, NOT} の組合せで表現できたと仮定して,  $(n + 1)$  入力の関数が表現できることをいう

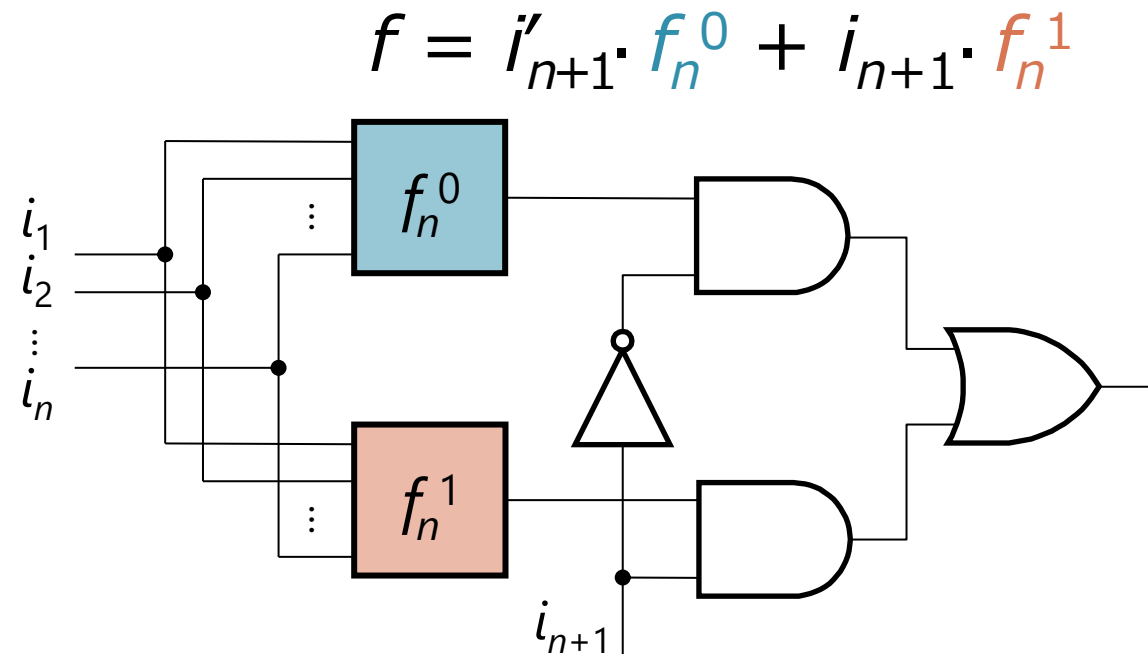


# 完全性の証明 {AND, OR, NOT}

## ■ 数学的帰納法：

1. 1入力の論理関数は {AND, OR, NOT} の組合せで表現できる
2.  $n$  入力の関数を {AND, OR, NOT} の組合せで表現できたと仮定して,  
( $n + 1$ ) 入力の関数が表現できることをいう

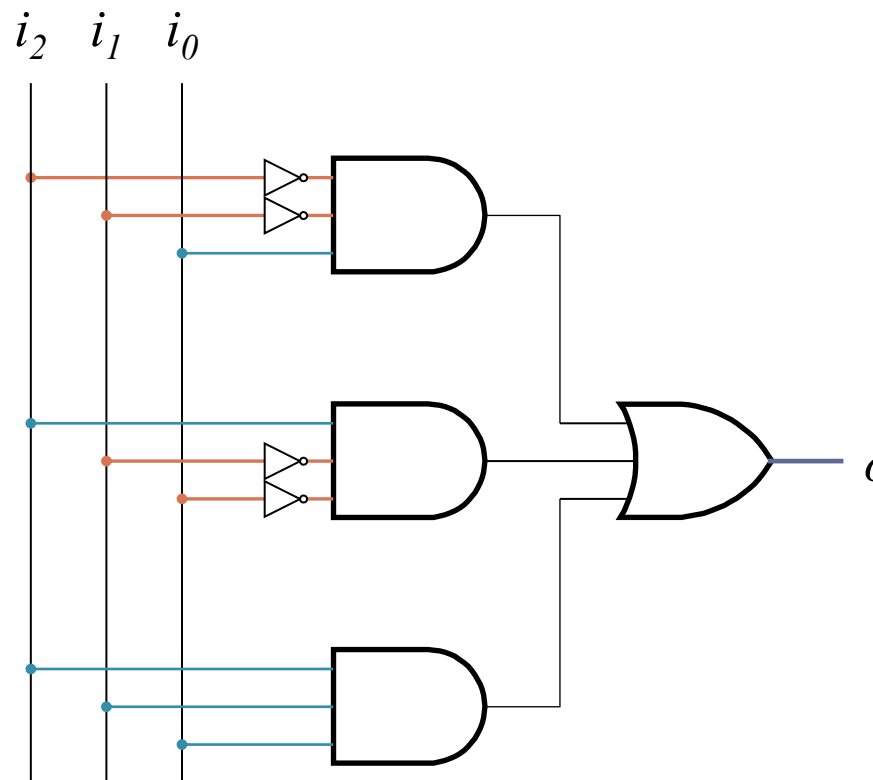
| $i_{n+1}$ | $i_n$    | ...      | $i_2$    | $i_1$    | $o$ |
|-----------|----------|----------|----------|----------|-----|
| 0         | 0        | ...      | 0        | 0        |     |
|           | 0        | ...      | 0        | 1        |     |
|           | $\vdots$ | $\vdots$ | $\vdots$ | $\vdots$ |     |
|           | 1        | ...      | 1        | 1        |     |
| 1         | 0        | ...      | 0        | 0        |     |
|           | 0        | ...      | 0        | 1        |     |
|           | $\vdots$ | $\vdots$ | $\vdots$ | $\vdots$ |     |
|           | 1        | ...      | 1        | 1        |     |



- 任意の論理関数  $f_n^0, f_n^1$  が表現できるので、それらの結果を  $i_{n+1}$  を使って選択
- この選択部分は AND/OR/NOT で作れる

# 真理値表による表現と，積和標準系による回路

| $i_2$ | $i_1$ | $i_0$ | $o$ |
|-------|-------|-------|-----|
| 0     | 0     | 0     | 0   |
| 0     | 0     | 1     | 1   |
| 0     | 1     | 0     | 0   |
| 0     | 1     | 1     | 0   |
| 1     | 0     | 0     | 1   |
| 1     | 0     | 1     | 0   |
| 1     | 1     | 0     | 0   |
| 1     | 1     | 1     | 1   |



$$o = i_2' i_1' i_0 + i_2 i_1' i_0' + i_2 i_1 i_0$$

■ 真理値表が与えられれば，

- {AND, OR, NOT} を使った積和標準系に機械的に置き換えできる
- つまり，{AND, OR, NOT} を使って対応する回路が生成できる



# 回路の例：RISC-V の AND/OR/XOR 命令の演算

XOR:  $x[rd] \leftarrow x[rs1] \wedge x[rs2]$

|         |     |     |            |    |         |
|---------|-----|-----|------------|----|---------|
| 0000000 | rs2 | rs1 | <u>100</u> | rd | 0110011 |
|---------|-----|-----|------------|----|---------|

OR:  $x[rd] \leftarrow x[rs1] \vee x[rs2]$

|         |     |     |            |    |         |
|---------|-----|-----|------------|----|---------|
| 0000000 | rs2 | rs1 | <u>110</u> | rd | 0110011 |
|---------|-----|-----|------------|----|---------|

AND:  $x[rd] \leftarrow x[rs1] \wedge x[rs2]$

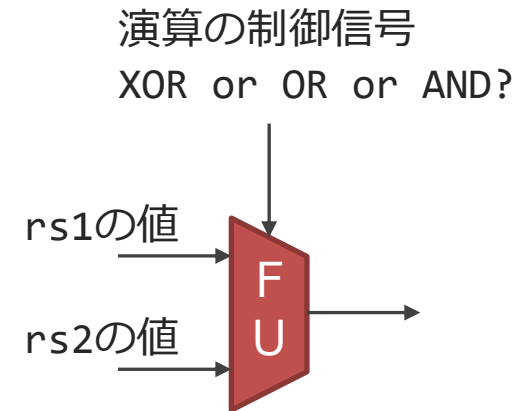
|         |     |     |            |    |         |
|---------|-----|-----|------------|----|---------|
| 0000000 | rs2 | rs1 | <u>111</u> | rd | 0110011 |
|---------|-----|-----|------------|----|---------|

## ■ RISC-V の AND/OR/XOR 命令

- まず右端の opcode が 0110011 であれば、この3つのどれかということにする
  - 本当はほかの命令との識別がさらにあるが、ここでは忘れる
- 真ん中の赤い3ビットの違いで識別する

# 制御の例：RISC-V の AND/OR/XOR 命令

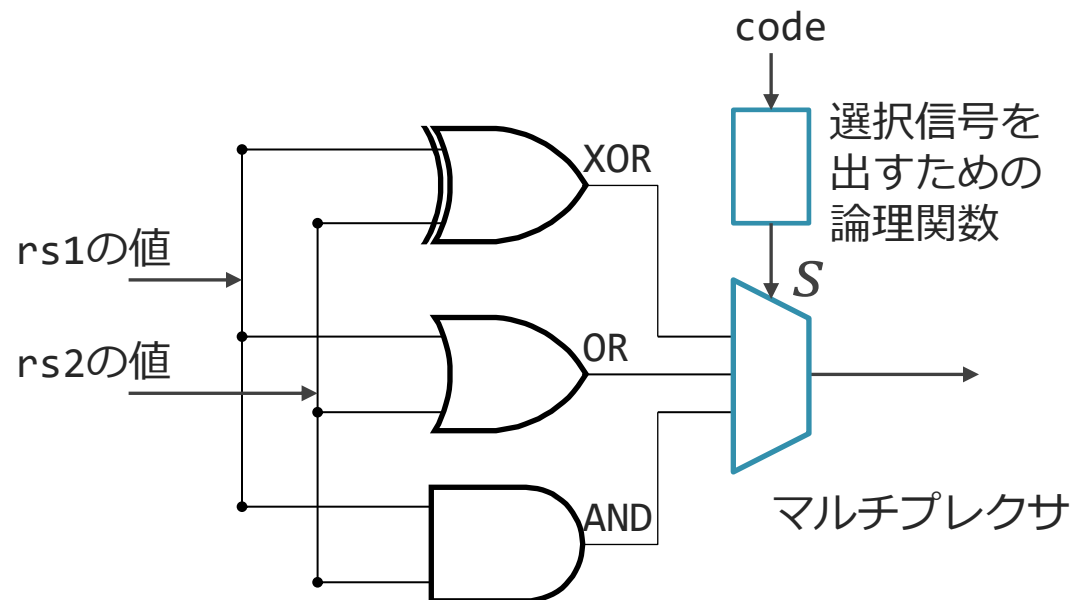
```
if code == 0b100:
 rs1 xor rs2
elif code == 0b110:
 rs1 or rs2
else:
 rs1 and rs2
```



- 各命令の 3bit の code の違いに応じて
  - 演算器に, AND/OR/XOR 演算をさせることを考える

# 制御の例：RISC-V の AND/OR/XOR 命令

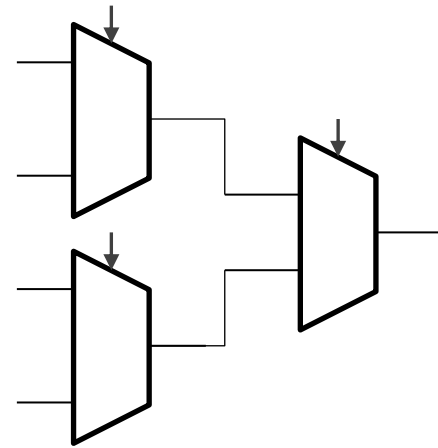
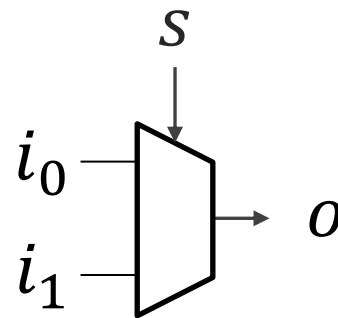
```
if code == 0b100:
 rs1 xor rs2
elif code == 0b110:
 rs1 or rs2
else:
 rs1 and rs2
```



- 典型的には,
  - 各場合ごとの回路を用意して並列に配置
    - XOR, OR, AND ゲートを並べる
  - 制御に従ってマルチプレクサで出力を選択

# マルチプレクサ：複数入力から1つを選ぶ回路

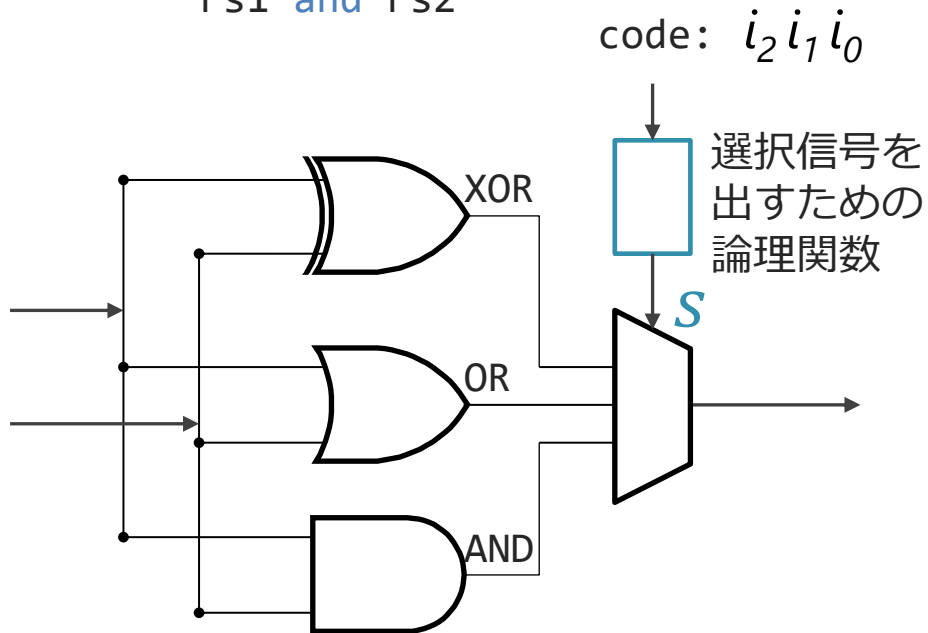
| $s$ | $i_0$ | $i_1$ | $o$ |
|-----|-------|-------|-----|
| 0   | 0     | 0     | 0   |
| 0   | 0     | 1     | 0   |
| 0   | 1     | 0     | 1   |
| 0   | 1     | 1     | 1   |
| 1   | 0     | 0     | 0   |
| 1   | 0     | 1     | 1   |
| 1   | 1     | 0     | 0   |
| 1   | 1     | 1     | 1   |



- 以下により，回路が生成できる
  - 2 : 1 マルチプレクサは真理値表でかける = 回路が作れる
  - 多入力マルチプレクサは，カスケードすれば良い

# 選択信号を出すための論理関数

```
if code == 0b100:
 rs1 xor rs2
elif code == 0b110:
 rs1 or rs2
else:
 rs1 and rs2
```

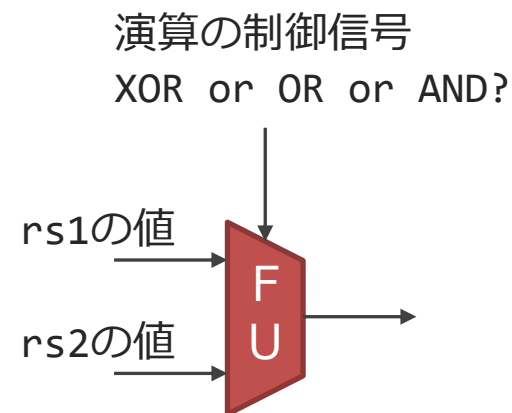


|     | $i_2$ | $i_1$ | $i_0$ | $S_1$ | $S_0$ |
|-----|-------|-------|-------|-------|-------|
|     | 0     | 0     | 0     | x     | x     |
|     | 0     | 0     | 1     | x     | x     |
|     | 0     | 1     | 0     | x     | x     |
|     | 0     | 1     | 1     | x     | x     |
| XOR | 1     | 0     | 0     | 0     | 0     |
|     | 1     | 0     | 1     | x     | x     |
| OR  | 1     | 1     | 0     | 0     | 1     |
| AND | 1     | 1     | 1     | 1     | 0     |

- 場合わけの制御は、そのまま真理値表にして回路を生成すればよい
  - XOR なら 00, OR なら 01, その他は 10

# 回路の生成のまとめ

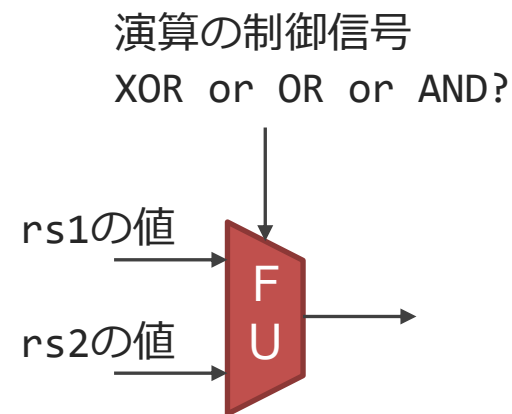
```
if code == 0b100:
 rs1 xor rs2
elif code == 0b110:
 rs1 or rs2
else:
 rs1 and rs2
```



- 結局この手の, 「条件に応じて異なる演算を出力する回路」は,
  1. 各場合ごとの回路を用意して並列に配置
  2. 制御に従ってマルチプレクサで出力を選択
- ……というように分解すれば, AND/OR/NOT 回路に落とし込める

# 回路の生成のまとめ

```
if code == 0b100:
 rs1 xor rs2
elif code == 0b110:
 rs1 or rs2
else:
 rs1 and rs2
```



- 原理的には, code, rs1, rs2 を全て含む真理値表を作れば, そこから直接回路に落とし込むこともできる
  - 表が大きくなりすぎて (2 の 3+32+32乗), 現実的には無理

# 組み合わせ回路と順序回路

## 1. 組み合わせ回路

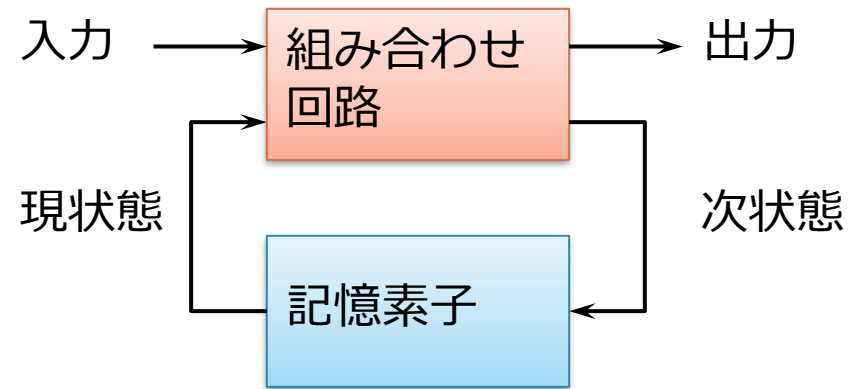
- 出力が、現在の入力のみにより決定される論理回路

## 2. 順序回路

- 出力が、過去の入力（の履歴）にも依存する論理回路

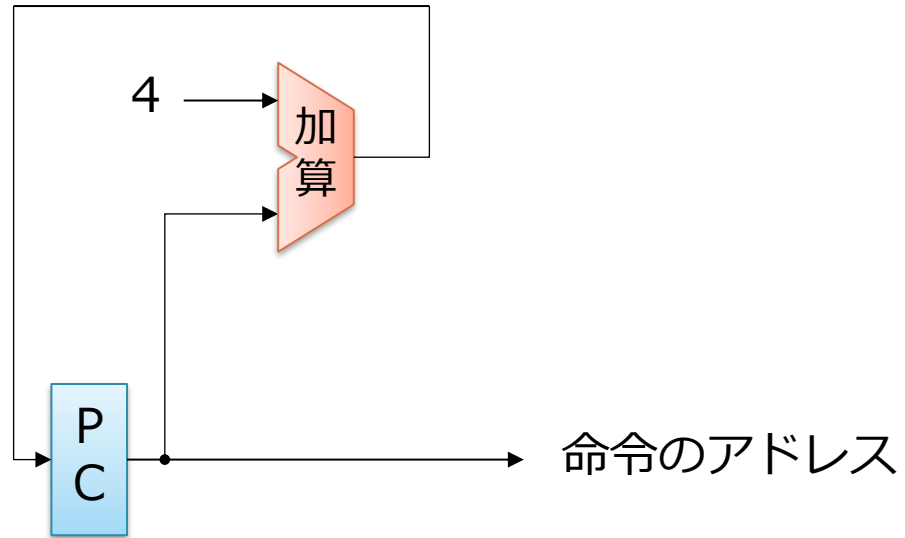


# 順序回路



- 出力が，過去の入力（の履歴）にも依存する論理回路
  - 記憶素子と，組み合わせ回路から成る

# CPU の PC 部分

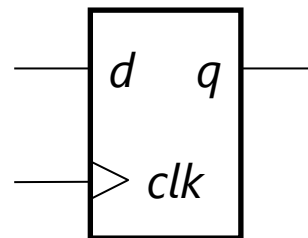


- 毎サイクル PC が加算される部分は，典型的な順序回路

# 記憶素子の例：D-FF（Flip Flop）

## ■ 入出力端子

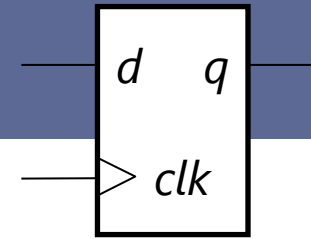
- ◇ データ入力 :  $d$
- ◇ データ出力 :  $q$
- ◇ クロック入力 :  $clk$



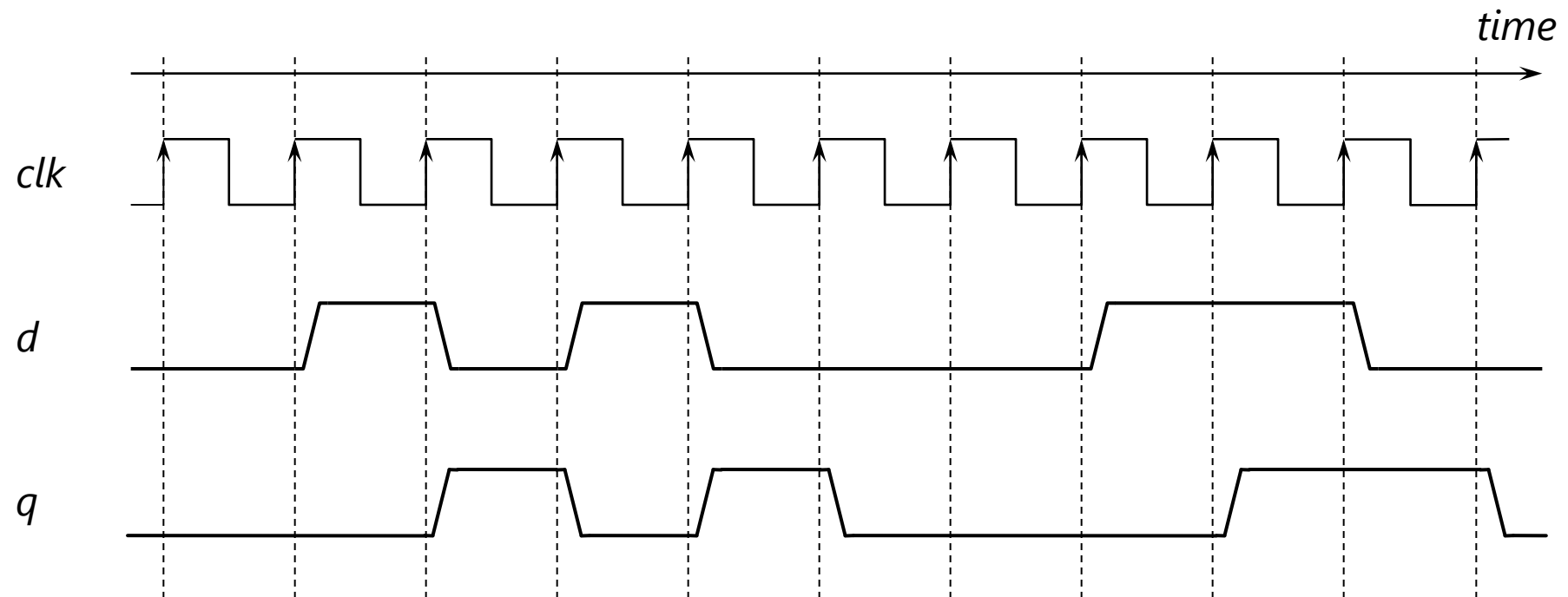
## ■ 働き：

- ◇ クロックの立ち上がりのたびに、 $d$  の値がサンプリングされる
- ◇ その値が次のサイクルの間  $q$  から出力される

# D-FF の動作



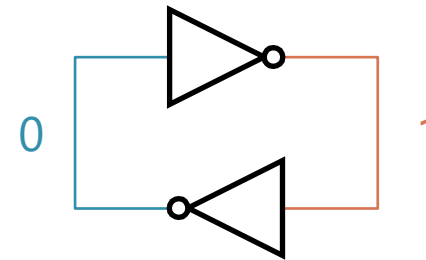
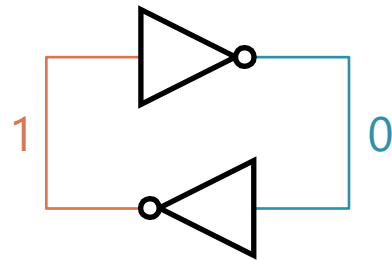
|          |   |   |   |   |   |   |   |   |   |   |
|----------|---|---|---|---|---|---|---|---|---|---|
| <i>d</i> | 0 | 1 | 0 | 1 | 0 | 0 | 0 | 1 | 1 | 0 |
| <i>q</i> | 0 | 0 | 1 | 0 | 1 | 0 | 0 | 0 | 1 | 1 |



# 記憶素子の原理

## ■ 記憶

- 2つの NOT ゲートをループさせた回路により記憶
- 2通りの安定状態がある：1 bit を記憶

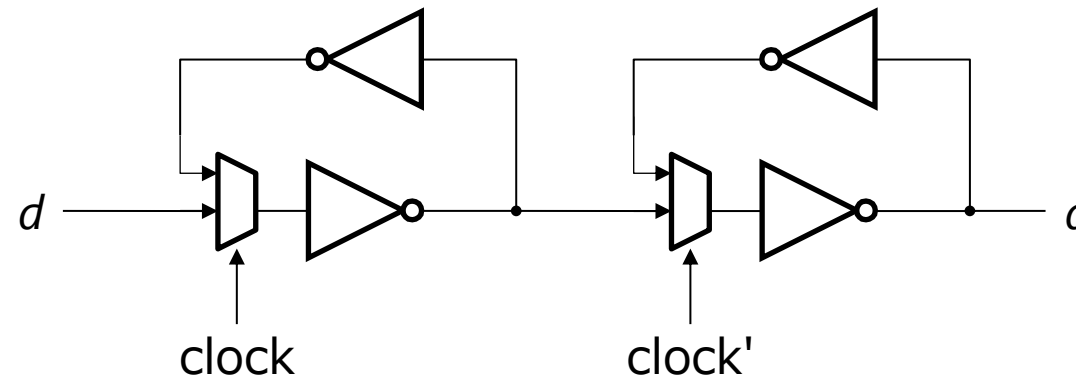


# D-FF の実装

## ■ 構造：

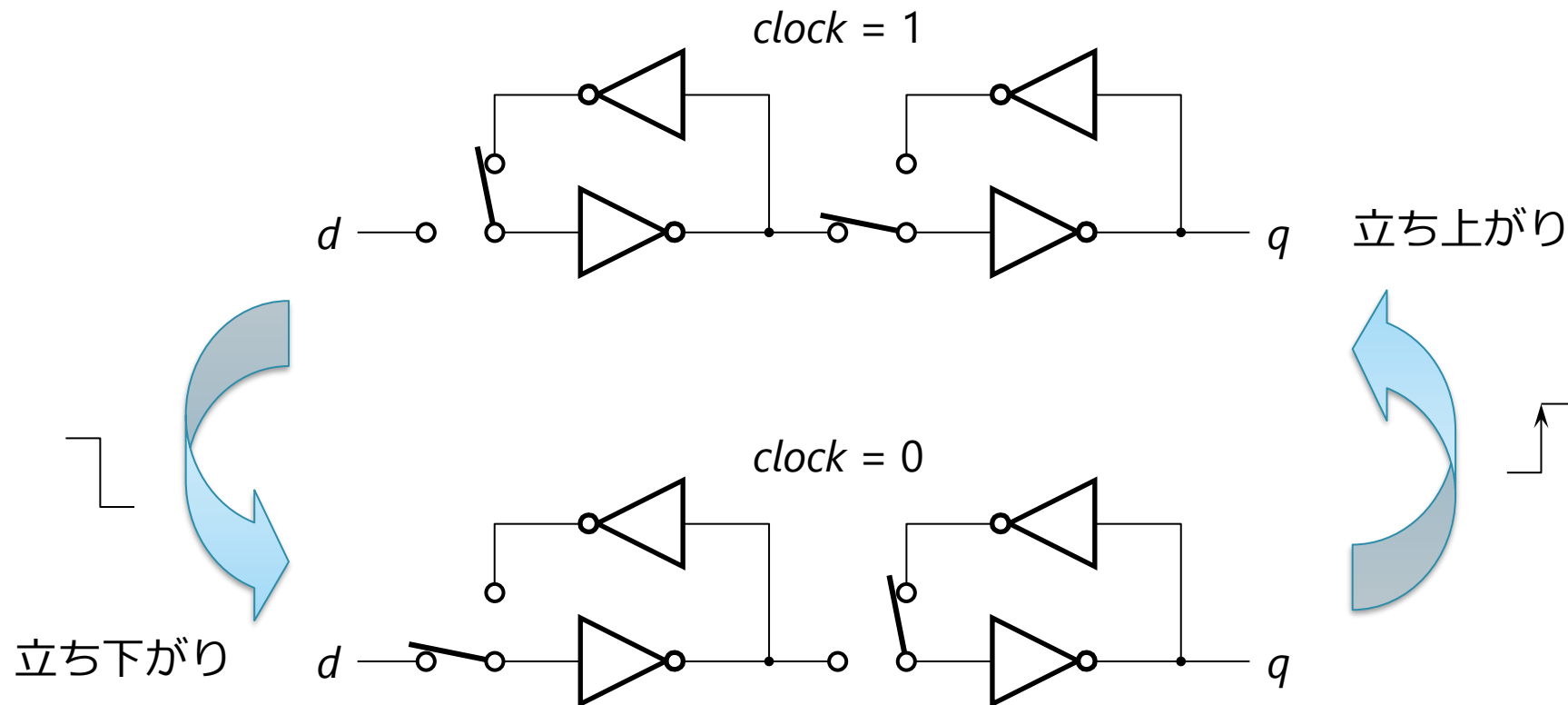
- ◇ NOT ゲートのループを二段に接続
- ◇ 各ループにはマルチプレクサが入っている

■ この構造は、クロックのエッジで記憶を更新するため

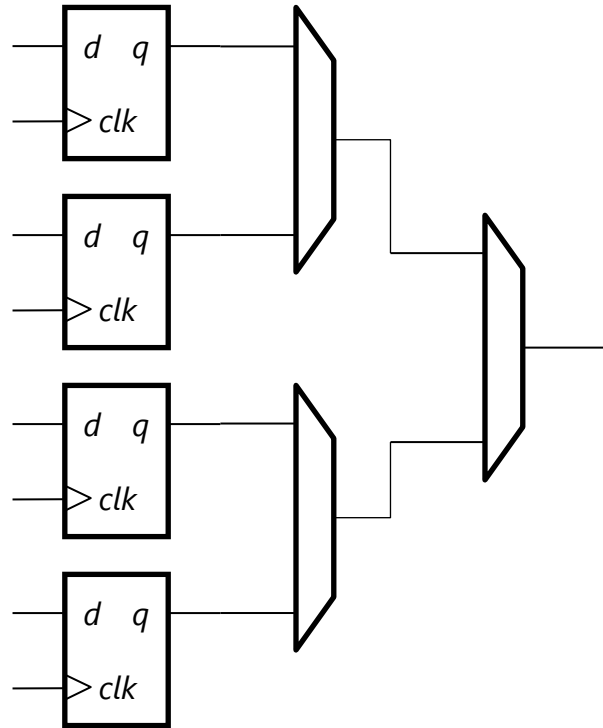


## D-FF の実現例

- マルチプレクサを, 切り替えスイッチとして説明
  - ◇ クロックの立ち上がりのたびに,  $d$  の値がサンプリングされる
  - ◇ その値が次のサイクルの間  $q$  から出力される



# メモリやレジスタ・ファイル



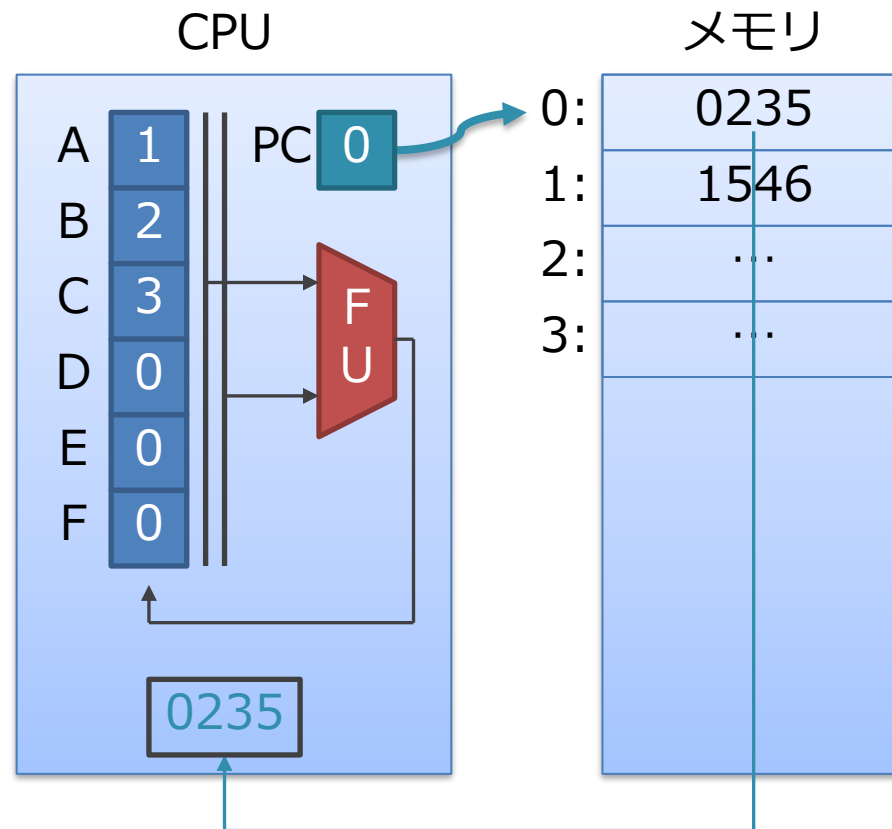
- D-FF を必要なだけ並べて，マルチプレクサで選択することで実現できる
  - 実際には，もっと最適化された回路（SRAM）が使用される事が多い
  - （後の講義で詳しく説明する予定）



# 組み合わせ回路/順序回路のまとめ

- 記憶素子も， NOT ゲートなどの論理ゲートで構成される
- 結果として， 任意の組み合わせ回路/順序回路は
  - 原理的には， 完全集合の要素の組み合わせに落とし込める
  - たとえば {AND, OR, NOT} を組み合わせれば， 全部つくれる

# これまでに説明した CPU の要素は、全てこれでカバー可能



# まとめ

1. 2進数や16進数による数値表現
2. 論理回路による実装
  - CPUの論理的な動作と、それを実現する物理的な回路の繋がり
  - この講義資料では（ていうか、今後の資料も結構）、一部、五島先生の「デジタル回路」の講義資料の図を使用しています

# 課題

1. 感想や質問を投稿してください
    - Moodle の「感想や質問」のところからお願いします
  2. 以降で説明する課題を「課題」に提出してください
    - 課題は自力でがんばって提出することに意義があります
    - 途中で力尽きてもある程度頑張った様子があれば、別に正解していなくても満点がつきます
- 締め切りは次回講義の開始時です
  - 生成 AI で課題の結果を生成したりしないでください
    - AI に聞くのはありますが、かならず理解してください

# 課題 3

- 以下では 2進数は 0b... 16進数は 0x... と表記するものとする
- 以下の 16進数を 2進数で表記せよ
  - (1) 0x1011
  - (2) 0x1234abcd
- 以下の 2進数を 16進数で表記せよ
  - (3) 0b101011
  - (4) 0b1111100001111
- 以下の演算を行え
  - (5)  $0b11011010 / 0b10$  (「/」 は割り算)

感想や質問

---

- 番地は適当に空いているところを選んだだけで番地の数字そのものに意味がない、というのを番地は何でもいいからCPU内でどのような処理がされるかが分かっていたらいい、という風にとったので、課題2 (2) ではアセンブリ言語のプログラム1行ごとに系列を書いたんですがそれで大丈夫ですか？
- また、少し調べたところメモリの中で命令用と数値（データ）用の場所（他にもヒープ？やスタックというものの用の場所も）が分かれていることを知り、それによって実行時の混乱が防がれているのだなと納得できました。

- ld命令が演算器を介してメモリの場所を読み込んでいるのに対し、st命令が演算器を介さず書き込んでいるのはなぜでしょうか。



- RISC-Vの話で、R-typeは何となく分かったのですが、I-typeの話が理解できませんでした。たとえばADDIなら、25～31ビット部分のfunct7の値？を演算に使っているようですが、他のI-typeの命令とどこで区別しているのでしょうか？

- "LABEL:" の後に書いたものがLABELの中身なのかそうでないのかを判断する基準はありますか。それともLABELの中身という認識自体が間違っていて、"LABEL:"はただジャンプする際の目印に過ぎないということですか。

- リアクションペーパーが苦手なのですが、この講義では感想質問が浅いと減点などありますか？それとも何かしら書いて出してさえいれば大丈夫ですか？

- 先輩から毎回の課題ができればテストも大丈夫だと聞いたのですが、課題以外の範囲も復習必須でしょうか？スライドが多く時間も多くはかけられないのでどこまで復習するか悩みます

- 課題もとても難しく、くじけそうです...

- 命令セットアーキテクチャが違っていると互換性がないとのことですが、例えばMacとWindowsの命令セットアーキテクチャは異なるということでしょうか？  
そうならば、互換性のないコンピュータにて同じプログラミング言語を使えるって凄いことですね.....！

- LABEL: という表記以外は左詰めではなく空白を作るといった決まりはありますか？

- 授業資料に乗っているアセンブリ言語の例は公式に使われている文法ですか？



- もし授業中に説明されていた内容でしたらすみません。  
今回のスライドのp.53とp.77,78の「LABEL」は、同じ働きをするものなのでしょうか。  
書き方を見ると、p.53はC言語でp.77,78はアセンブリ言語のように感じました。  
どちらの言語でもラベルは使えるのでしょうか？  
その場合、条件文を書く際には、LABELを使うかアドレスを直接打ち込むかどちらを使うべきか教えていただきたいです。

# 質問や感想など

- 定義したラベルが前の方にありその後ろにそのラベルに飛ぶという命令がある時（ループするやつ）、一周目のラベルの定義の中身は実行されるのでしょうか？それとも、ラベルに飛ぶという指示が出るまでラベルの中身は実行されないのでしょうか？
- また、課題はループしないようでしたのでこのように提出したのですが、ラベルに飛ぶという指示の後にラベルの定義を書くのは問題ないのでしょうか？

- アセンブリ言語に入っている幾つかの命令から、さまざまな処理を維持できる点がおもしろかった。Program Counterの略称PCは、無意識にPersonal Computerと呼んでしまうので混乱した。

# 質問や感想など

- ① LABELがついているメモリ番地から下の番地は、ジャンプしない限りは読まれないのですか。それによって(2)の書き方が変わってくると思いました。
- ② (1)で LABEL とそれに続く式を書くべき位置がわからないのですが、もしかしてどこでもいいのですか。
- ③ `li 0x420 -> B` ってこんなにたくさん書く必要ありますか。
- ④ メモリに書いてある命令はどうなったら終了なのですか。 `end` とかを書くのでしょうか。
- また LABEL があるところとその下の式は if 文が `false` の時でもスキップしないで読まれてしまうのですか。
- ⑤ if 文が真の時、何も処理をしなくても値は変わらないので、何も処理をしないというのは、プログラム通りではないですがメモリの節約にはなるので良いとは思ったのですが実際はどうなのでしょう。

- アドレスやポインタに対して若干の苦手意識があるのですが、この授業でコンピュータの中身について知っていくことで二重ポインタや三重ポインタも臆することなく使えるようになりたいと思いました。

- カレーのレシピで例えるのが分かりやすかったです。用語をどこまで覚えないといけないのかが気になってます。

- アセンブリ言語の名前の由来などがありますか？

- アセンブリ言語という存在を初めて知りました。C言語は1年生の頃から習ってきたが分かりにくいことも多くありました。アセンブリ言語は読みやすくて起こっていることを把握しやすく、人間とコンピュータを繋ぐと感じられました。アセンブリ言語をはじめに習いたかったです。



# 質問や感想など

- 質問,感想:アドレスとポインタの話がとても難しかったです。プログラムを書いているときもどこにアクセスしているかわからなくなってしまうことが多いので復習します。期末テストは今回のアドレスなどの内容がわかってないとまずいですか？頑張ります。

- スライド48頁のiについて、グローバル変数としている理由を知りたいと思いました。ローカル変数だと不都合があるのでしょうか。

- 用語を問われるような問題も期末テストに出ますか。それとも今回の課題のような問題が中心ですか。

- C言語のループの置き換えについて、Bと(B)の違いは何でしょうか？

- 普段プログラミングをする際にはC言語やPython、Rなどを用いてコードを書くが、それらを全てアセンブリ言語に置き換えて記述することは可能なのかと、実際にアセンブリ言語はどこで使用されているのか  
が疑問に思った。

- 「機械語⇒バイナリないしはアセンブリ言語で表現されたプログラム」というのが腑に落ちませんでした。アセンブリ言語と機械語は区別されるべきものではないのでしょうか。

- バイナリはコンピュータが直接理解できる言語だという説明があったが、逆に人間が翻訳などなくバイナリだけを解読することは可能なのか気になった。

- RISC-Vのような命令セットの設計思想（RISC）において、命令をあえて単純化することで処理効率上が上がるのはもちろんだが、それがもたらす最大のメリットは何か気になった。



- 条件分岐では「条件が偽のときに分岐する」ように逆の条件で書くことが多いと感じたが、これはハードウェア的な制約によるものなのか、それとも効率の観点からそうしているのかが気になった。